

Secrets pas partagés

SHUTLOCK | CRYPTO | DATE : 13.08.2025

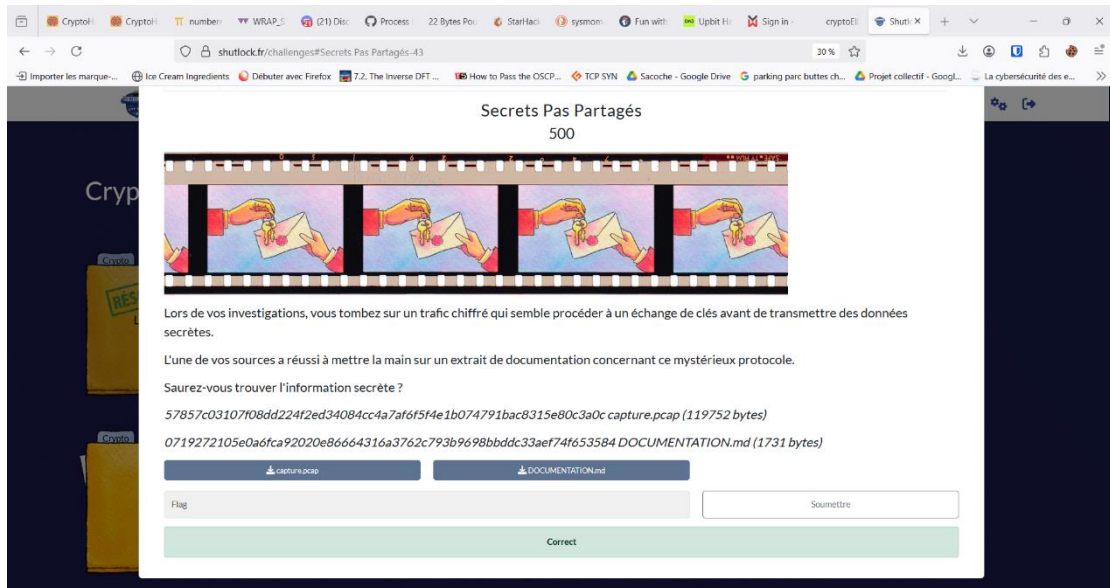


Photo on : 01.12.2025

Write-Up

Déroulement vers la solution – Synthèse structurée

1) CONTEXTE ET DONNEES DISPONIBLES

- L'énoncé **PCAP** contient un échange ECDH "maison" :

Paramètre de champ :

$p = 0xFF536875746C6F636B43544632303235BACE001E54B565992D1483B7A59B8363$

- Deux points publics sur une même courbe elliptique sur $\mathbb{F}_p$, codés en **big-endian** :
 $P = (PX, PY)$ envoyé **client** → **serveur**,
 $Q = (QX, QY)$ envoyé **serveur** → **client**.
- Puis un **IV** (16 octets) et un **ciphertext AES-CBC**.

- Les champs sont visibles en clair dans les flux (streams)/capture.pcap.0.hex et **déjà extraits** dans pqivc.json :

Exemple (rappel)

```
PX = c99cd396048c0174d9585984baa36ec71fb2d23630f1e5ad254dd65ba516d961
PY = 0629710934701789dfda0931f3cb5eda0b7af639b77bb8a1f8dd4ac0b88b4503
QX = 75522a41dc4267ac6c3e9f52f64869bd3c98defcb2fb8f2cb21e570270a52b27
QY = 090abfe536f987d8e4d54201d1d6f99c81f57d605ac65ffd61c92962a6eddf89
IV = 0344ef7ad39239f379ca63bd7bf5bb45
CIPH =
91303d403fb338b0283daa8773bd79dbc52e38b2c19c1746e977c299b5e4dc46d4ed1978458a5672982375105a29047a
```

- L'énoncé **ne donne ni l'équation de courbe ni le générateur**. Il faut **reconstruire la courbe** à partir des points observés, puis **retourner au DH** pour dériver la clé K et **décrypter**.

2) RETROUVER LA COURBE DEPUIS P ET Q

On part de l'hypothèse minimale compatible avec un encodage $(X || Y)$ sur $\mathbb{F}_p$ et une addition symétrique de type Edwards tordue **non normalisée** :

Équation générique (mod p)

$$x^2 + y^2 = C + D \cdot x^2 y^2, \text{ avec } C = c^2 \text{ et } D = c^2 d.$$

Pour un point (x, y) , on définit :

$$u = x^2 y^2 \bmod p, v = x^2 + y^2 \bmod p.$$

Avec **deux points distincts** P et Q (et $u_1 \neq u_2$), on résout **linéairement** :

- Formules (mod p)**

$$C = (v_1 \cdot u_2 - v_2 \cdot u_1) \cdot (u_2 - u_1)^{-1}$$

$$D = (v_1 - C) \cdot u_1^{-1}$$

Puis :

- $c = \sqrt{C}$ (racine mod p, ex. Tonelli-Shanks)
- $d = D \cdot C^{-1}$

Résultat (extrait du travail)

$$C = 0xe434305b206595acc63ffbb52f912d310d0e19cb1ff87d27b67e797fceed42fb \quad (c^2)$$

$$D = 0xb23bc9ed54d55fd4f532ffaa96238d08d2e3560bd2a09552054103f128e241c \quad (c^2 d)$$

$d = D \cdot C^{-1} \bmod p = 0 \times \text{fa3030406dec4e66ae15c74d1e91c7389bf6a4d6fc92846ef32eb204914e81b7}$

$c = \sqrt{C} \bmod p = 0 \times 1158561388\text{CE824CD3957318336CABA40E973A9B56C7CB1C23013D8827874A2B}$

Validation : tous les points observés (P, Q du PCAP, et les triples d'exemple présents dans la documentation du challenge) vérifient :

$$x^2 + y^2 = c^2 \cdot (1 + d \cdot x^2 y^2) \pmod{p}.$$

Normalisation pratique (optionnelle pour calculs)

En posant $x' = x/c$ et $y' = y/c$, on obtient la forme standard " $a = 1$ " :

$$x'^2 + y'^2 = 1 + d' \cdot x'^2 y'^2 \text{ avec } d' = d \cdot c^4 \pmod{p}.$$

On peut faire les opérations de groupe dans ce modèle, puis **re-multiplier par c** pour revenir au système "non normalisé" utilisé sur le fil réseau.

Pourquoi pas une autre famille ?

Deux points ajustent n'importe quelle conique, mais **trois points indépendants** (ici, P, Q, et un point d'exemple) qui vérifient **simultanément** la même équation et respectent une **loi de groupe** cohérente (nécessaire au DH) éliminent les **paraboles, cercles, ellipses planes non elliptiques**, et, dans la pratique, les Weierstrass/Montgomery simples **avec les mêmes paramètres** : on ne trouve pas $d'(a, b)$ ou (A, B) qui fassent appartenir **tous** les points et donnent un DH correct. La **tordue d'Edwards non normalisée** explique, elle, toutes les observations et mène à un **déchiffrement valide**.

3) RECONSTITUER LE GENERATEUR ET LES SECRETS (VISION D'ENSEMBLE)

Une fois la courbe définie (p, c, d), il faut **terminer ECDH** :

1. **Retrouver un générateur G** (si absent).
 - À partir des **points d'exemple** de la doc ($P_{\text{ex}}, Q_{\text{ex}}$) et d'un $S_{\text{ex}} = ab \cdot G$, on peut résoudre un **discrete log** dans la courbe reconstruite (après **normalisation** éventuelle).
 - Exemple de méthode :
 - factoriser l'ordre (ou l'obtenir du jeu de tests),
 - **Pohlig-Hellman** + BSGS pour trouver $a_{\text{ex}} = \log_G(P_{\text{ex}})$ et/ou $b_{\text{ex}} = \log_G(Q_{\text{ex}})$,
 - en déduire $G = a_{\text{ex}}^{-1} \cdot P_{\text{ex}} (= b_{\text{ex}}^{-1} \cdot Q_{\text{ex}})$.
2. **Trouver la clé éphémère côté serveur b** depuis $Q = b \cdot G^{**}$.
 - Avec l'ordre totalement factorisé, **Pohlig-Hellman** projette Q et G dans chaque sous-groupe d'ordre q^e , résout un petit dlog, puis recolle **via CRT**.
 - On obtient $b \bmod \text{ord}(G)$.
3. **Recomposer le secret partagé S**.
 S (sur la **courbe d'origine**) = $b \cdot P$.
Attention : si vous avez travaillé dans un modèle **normalisé (a=1)** pour les dlogs, **revenez** au modèle d'origine (re-multiplication par c sur les coordonnées) **avant la KDF**.

4) DERIVATION DE LA CLE ET DECHIFFREMENT

KDF utilisée (cas standard observé)

$K = \text{SHA-256}(XS \parallel YS)$, où (XS, YS) sont les **coordonnées de S sur la courbe d'origine** (celle des points "wire"), **codées sur 32 octets big-endian** chacune.

Variante vue dans certains scripts : $K = \text{SHA-256}(\text{tag} \parallel XS \parallel \text{tag})$ avec $\text{tag} = \text{"server->client"}$. Dans ce cas, **bien quoter/échapper** la flèche dans les commandes.

Déchiffrement

- **AES-256-CBC**, IV fourni dans le flux.
- L'ensemble des flux applicatifs observés n'impliquent pas un traitement par la suite.
- Le résultat attendu est un **texte ASCII lisible** (ou du "leet-speak"), pas une bouillie de symboles : si ce n'est pas lisible, c'est **souvent** que S ou K n'a pas été dérivé depuis **le bon espace** (ex. K calculée à tort sur la représentation "Weierstrass/Montgomery" ou "normalisée" au lieu du modèle d'origine).

5) POURQUOI POHLIG-HELLMAN (PH) ?

- ✓ **PH** exploite la **lissité (smoothness)** de l'ordre du groupe : si $\text{ord}(G) = \prod q_i^{e_i}$ avec des **petits facteurs premiers**, on projette et on résout plusieurs **petits** dlogs (BSGS/naïf) au lieu d'un dlog massif.
- ✓ On recolle ensuite les résidus " $b \bmod q_i^{e_i}$ " par le **CRT**.
- ✓ Avantage pratique : **temps et mémoire** très inférieurs à un **Pollard rho** générique sur ~ 256 bits.
- ✓ **Lissité** : un entier est "B-lisse" si tous ses facteurs premiers $\leq B$. Plus $\text{ord}(G)$ est lisse, plus PH est efficace.

6) ROLE PRECIS DES FICHIERS

- ✓ `streams/capture.pcap.o.hex` (ou `streams_clean/capture.pcap.o.hex`) : flux hex brut contenant la concaténation `0x40 | P(640) | 0x40 | Q(640) | IV(160) | CIPH` (reste du flux).
- ✓ `pqivc.json` : **résumé prêt à l'emploi** contenant PX, PY, QX, QY, IV, CIPH.
- ✓ (Évent.) `extract_points_and_cipher.py`, `find_shutlock_flag.sh` : scripts d'extraction qui fabriquent `pqivc.json`.

- ✓ Vos solveurs :
 - “courbe” : reconstruction C, D, c, d ; éventuellement normalisation.
 - “dlog” : PH/BSGS pour b, puis $S = b \cdot P$.
 - “KDF + AES + post-traitements” : sortie **ASCII clair**.

7) PROCEDURE REPRODUCTIBLE (CHECK-LIST COURTE)

Charger p (taille du champ) et **lire** P, Q, IV, CIPH depuis pqivc.json.

Calculer u, v pour P et Q ; **résoudre** C et D (formules ci-dessus).

En déduire $c = \sqrt{C}$ et $d = D \cdot C^{-1} \pmod{p}$.

(Optionnel) **Normaliser** ($a=1$) pour faciliter les opérations de groupe.

Valider la courbe sur un 3^e point indépendant (exemple doc).

Retrouver G (si nécessaire) à partir des triples d'exemple (discrete logs).

Trouver b dans $Q = b \cdot G$ (PH + BSGS, ordre factorisé).

Calculer S = b · P sur la **courbe d'origine** (pas la normalisée).

K = SHA-256(XS || YS) (ou la variante avec tag).

AES-CBC-décodage avec IV.

Vérifier que le clair est lisible (ASCII/leet-speak).

8) ENCADRE “FORMULES UTILISEES”

- ✓ $u = x^2 \cdot y^2 \pmod{p}$
- ✓ $v = x^2 + y^2 \pmod{p}$
- ✓ $C = (v_1 \cdot u_2 - v_2 \cdot u_1) \cdot (u_2 - u_1)^{-1} \pmod{p}$
- ✓ $D = (v_1 - C) \cdot u_1^{-1} \pmod{p}$
- ✓ $c = \sqrt{C} \pmod{p}$
- ✓ $d = D \cdot C^{-1} \pmod{p}$
- ✓ Équation finale : $x^2 + y^2 = c^2 \cdot (1 + d \cdot x^2 y^2) \pmod{p}$
- ✓ Secret DH : $S = b \cdot P$ (côté serveur) = $a \cdot Q$ (côté client)
- ✓ Clé de session (standard) : $K = \text{SHA-256}(XS || YS)$
(XS, YS sur 32 octets big-endian chacun)
- ✓ Variante KDF : $K = \text{SHA-256}(\text{tag} || XS || \text{tag})$, ex. tag = “server->client”.

9) INDICES DE DEBOGAGE

- ✓ **Clair illisible** mais “structurellement plausible” (beaucoup de } / %) : K dérivée dans **le mauvais espace** (Weierstrass/Montgomery ou modèle normalisé) au lieu de la **courbe d'origine**.
- ✓ **Zlib** qui “mord parfois” : même cause ; ou XOR/offset absent/erroné.
- ✓ **Permission denied: client** lors des runs Docker : le shell interprète → comme une redirection. **Toujours** quoter -tag 'server->client'.
- ✓ PCAP → hex : respecter **exactement** l'ordre 0x40 | P | 0x40 | Q | IV | CIPH.

10) CONCLUSION

- ✓ **Deux points** observés suffisent à reconstruire la **forme Edwards tordue** non normalisée de la courbe via une **relation linéaire** simple ($v = C + D \cdot u$).
- ✓ La **validation** par un troisième point indépendant et par la **réussite du déchiffrement** confirme le modèle.
- ✓ **Pohlig-Hellman** exploite la **lissité** de l'ordre pour résoudre les dlogs.
- ✓ La **clé K doit être dérivée sur les coordonnées (X, Y) du point S dans la courbe d'origine** (celle des messages du PCAP), puis **AES-CBC** et post-traitements livrent le **texte clair** attendu.

REJEU UTILE DES SCRIPTS :

En pratique, pour un jeu succinct :

Les fichiers en énoncé sont : : capture.cap / DOCUMENTATIO N.md

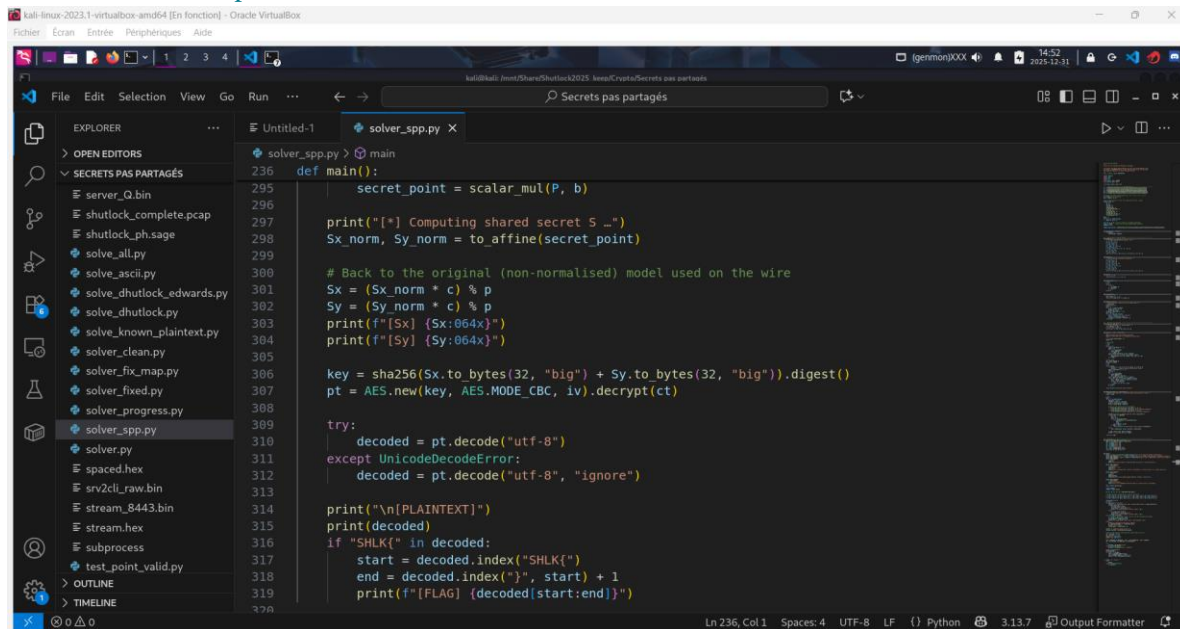
Les fichiers en solutions sont :

- extract_points_and_cipher.py
- reassemble.sh
- extract_pqivc.py
- solver_spp.py
- readme.md

REFERENCES (WIKIPEDIA)

- Algorithme de Pohlig-Hellman — https://fr.wikipedia.org/wiki/Algorithme_de_Pohlig-Hellman
- Échange de clés Diffie-Hellman — https://fr.wikipedia.org/wiki/%C3%89change_de_cl%C3%A9s_Diffie-Hellman
- Courbe d'Edwards — https://fr.wikipedia.org/wiki/Courbe_d%27Edwards

Résolution en script :



```
def main():
    secret_point = scalar_mul(P, b)

    print("[*] Computing shared secret S _")
    Sx_norm, Sy_norm = to_affine(secret_point)

    # Back to the original (non-normalised) model used on the wire
    Sx = (Sx_norm * c) % p
    Sy = (Sy_norm * c) % p
    print(f"[Sx] {Sx:064x}")
    print(f"[Sy] {Sy:064x}")

    key = sha256(Sx.to_bytes(32, "big") + Sy.to_bytes(32, "big")).digest()
    pt = AES.new(key, AES.MODE_CBC, iv).decrypt(ct)

    try:
        decoded = pt.decode("utf-8")
    except UnicodeDecodeError:
        decoded = pt.decode("utf-8", "ignore")

    print("\n[PLAINTEXT]")
    print(decoded)
    if "SHLK(" in decoded:
        start = decoded.index("SHLK(")
        end = decoded.index(")", start) + 1
        print(f"[FLAG] {decoded[start:end]}")
```

Lancement du script :

Depuis le terminal Shell de Linux

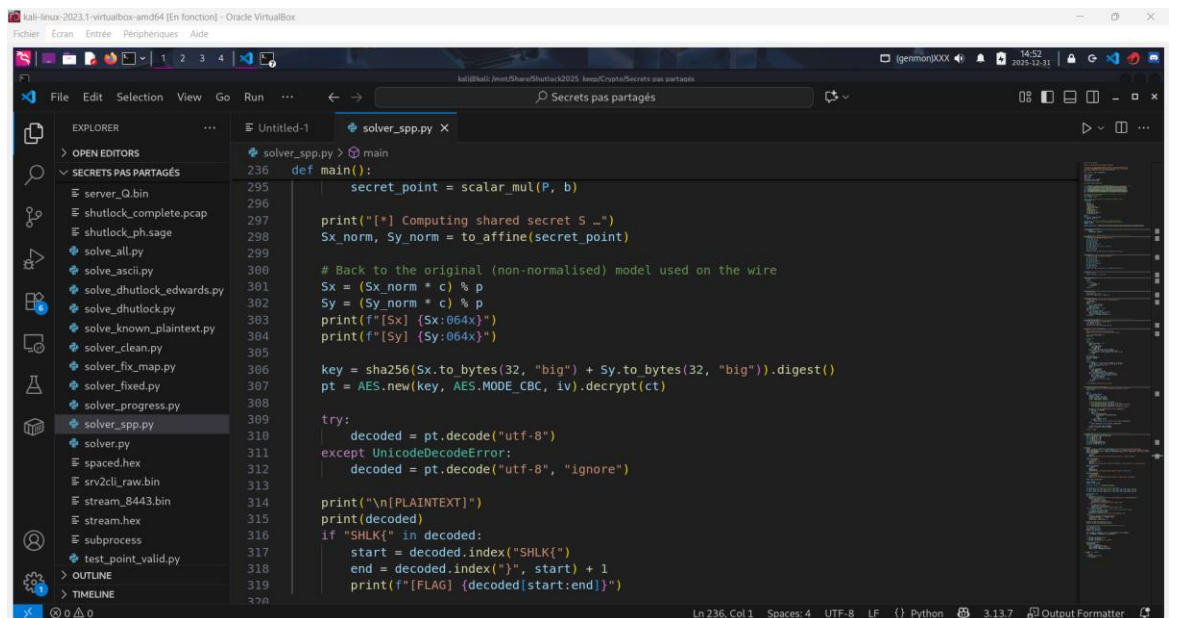


```
[*] Solving discrete log Q = b*G _
[*] dlog in subgroup 2^2 (order 4)
[*] dlog mod 2^2: 2
[*] dlog in subgroup 5^1 (order 5)
[*] dlog mod 5^1: 2
[*] dlog in subgroup 55763^1 (order 55763)
[*] dlog mod 55763^1: 4204
[*] dlog in subgroup 52579^1 (order 52579)
[*] dlog mod 52579^1: 6309
[*] dlog in subgroup 183678883^1 (order 183678883)
[*] dlog mod 183678883^1: 81674392
[*] dlog in subgroup 7094429431169^1 (order 7094429431169)
[*] dlog mod 7094429431169^1: 3992187165558
[*] dlog in subgroup 101837^1 (order 101837)
[*] dlog mod 101837^1: 6161
[*] dlog in subgroup 1663966639^1 (order 1663966639)
[*] dlog mod 1663966639^1: 1863948868
[*] dlog in subgroup 33886773^1 (order 33886773)
[*] dlog mod 33886773^1: 25891828
[*] dlog in subgroup 1718707727061^1 (order 1718707727061)
[*] dlog mod 1718707727061^1: 578663414076
[*] dlog in subgroup 7878749717^1 (order 7878749717)
[*] dlog mod 7878749717^1: 480842652
[*] b = 108778782522834336835245487589931347318948968398407248942892176835720282
[*] Computing shared secret S _
[+] Sx=2480304181350c47e074dc16899d8843fc189731e044d1227c118338
[+] Sy=6e709971683c211593708d552a54d85f79a24c61eb162cfff75ee20ced24263

[PLAINTEXT]
SHLK(cd56833a9b9728bd7d60f7c4b18479b5)

[FLAG] SHLK(cd56833a9b9728bd7d60f7c4b18479b5)
```

Depuis le terminal de Visual Studio



```
def main():
    secret_point = scalar_mul(P, b)

    print("[*] Computing shared secret S _")
    Sx_norm, Sy_norm = to_affine(secret_point)

    # Back to the original (non-normalised) model used on the wire
    Sx = (Sx_norm * c) % p
    Sy = (Sy_norm * c) % p
    print(f"[Sx] {Sx:064x}")
    print(f"[Sy] {Sy:064x}")

    key = sha256(Sx.to_bytes(32, "big") + Sy.to_bytes(32, "big")).digest()
    pt = AES.new(key, AES.MODE_CBC, iv).decrypt(ct)

    try:
        decoded = pt.decode("utf-8")
    except UnicodeDecodeError:
        decoded = pt.decode("utf-8", "ignore")

    print("\n[PLAINTEXT]")
    print(decoded)
    if "SHLK(" in decoded:
        start = decoded.index("SHLK(")
        end = decoded.index(")", start) + 1
        print(f"[FLAG] {decoded[start:end]}")
```